

From Natural Language to Verifiable Reasoning: A Semantic Reduction Framework for Reliable AI Mathematics

Anonymous Author(s)

C2 Challenge Submission — replace with name and affiliation for camera-ready
anonymous@example.com

October 2, 2026

Abstract

Large language models (LLMs) now produce fluent, apparently rigorous solutions to mathematical problems, yet they continue to output *plausible but invalid* proofs: a natural-language argument cannot be checked against a fixed formal semantics, so a mistaken step is indistinguishable from a correct one without re-doing the reasoning. The field has answered this reliability problem with three families of techniques: search with verification, formalized proof, and reproducible benchmarking. This paper argues that these answers share an overlooked structural feature. We decompose an AI-for-mathematics (A14MATH) system into three layers—*generation*, *semantic reduction*, and *verification*—and show that the reduction map from informal language to a computable, typed object is the decisive reliability bottleneck: when this map fails, the verifier has no object to inspect and hallucinations cannot, in principle, be detected; when it succeeds, verification is deterministic and can be made sound. On this basis we propose (i) a typed intermediate representation (IR) in which natural language serves only as an interface and the IR as the execution core, (ii) a generate→IR→verify→render pipeline with bidirectional consistency between text and structure, and (iii) a concrete ten-item reliability checklist for A14MATH systems. A small, fully reproducible reference implementation on a controlled word-problem dialect instantiates the framework: across 36 problem instances the verifiability rate was 0.833, every failure localized at the reduction layer, and kernel decisions on all 90 verifiable candidates were correct. We state explicitly what these results do and do not establish, and discuss applications to automated theorem proving, problem solving, and assessment.

1 Introduction

1.1 Background

Mathematics has become a flagship domain for modern AI. Systems built on large language models solve a large fraction of grade-school and competition problems on standard benchmarks such as GSM8K (Cobbe et al., 2021) and MATH (Hendrycks et al., 2021), while specialized systems have reached olympiad-level performance in geometry (Chervonyi et al., 2025; Trinh et al., 2024) and in formal reasoning (Hubert et al., 2025). Two technical moves drive this progress: eliciting intermediate reasoning steps through chain-of-thought prompting (Wei et al., 2022), and delegating parts of the computation to external tools or programs (Chen et al., 2023; Schick et al., 2023).

This success, however, coexists with a characteristic failure mode. An LLM emits a proof that *reads* as correct—every sentence is plausible, the notation is well formed, and the final answer may even be right—but contains an invalid inference hidden in the middle. Such failures are not rare anecdotes: performance varies sharply across numerically different instantiations of the *same* problem template, degrades as the number of clauses grows, and can collapse when a single irrelevant clause is added (Mirzadeh et al., 2024). Fluent text, in other words, is weak evidence of a valid argument.

1.2 The reliability problem

We call the central question the *reliability problem*:

Given an AI-produced mathematical argument, can an external party establish—without re-deriving the result—whether every inference is valid, and can that establishment be repeated by anyone else?

The literature offers three responses. **Search plus verification**: sample many candidate solutions and rank them with a learned or rule-based verifier (Cobbe et al., 2021; Lightman et al., 2024; Wang et al., 2023). **Formalized proof**: express the problem in a proof assistant such as Lean and have a small trusted kernel check the proof object (First et al., 2023; Hubert et al., 2025; Polu and Sutskever, 2020; Xin et al., 2024). **Reproducible evaluation**: pin down data, prompts, seeds, and templates so that reported numbers can be regenerated (Mirzadeh et al., 2024; Yang et al., 2023; Zheng et al., 2022).

These responses are usually presented as alternatives. We instead read them as instances of one underlying requirement: somewhere between the model and the checker, informal language must be converted into a *computable object with a fixed meaning*. This paper makes that conversion—which we term *semantic reduction*—explicit, studies where it fails, and builds a framework around it.

1.3 Thesis and contributions

Our thesis is:

Reliability in AI4MATH cannot be obtained by final verification alone, because verification can only inspect what reduction has already made into an object. Semantic reduction must therefore be a first-class, typed, and measurable system component.

We make four contributions, of which the last three are deliberately small and concrete:

1. **A three-layer decomposition** (generation, semantic reduction, verification) that reorganizes existing techniques and localizes the dominant failure mode (§2–§3).
2. **A typed-IR framework**: a pipeline architecture in which natural language is only an interface, together with a bidirectional consistency requirement between text and structure (§4).
3. **A ten-item reliability checklist** that operationalizes the framework for practitioners and reviewers (§4.4).
4. **A reproducible reference implementation and case study** that tests the central claim on a controlled dialect (§4.5); code and pinned seeds are included.

1.4 Scope and outline

The emphasis is on the structure of reliable systems and on AI-assisted scholarship, not on deep new mathematics. We do *not* claim that all informal reasoning can or should be formalized, nor that the small case study predicts benchmark performance; these boundaries are stated where the relevant claims are made. §2 defines the layers, §3 analyzes reduction as the bottleneck, §4 presents the framework and checklist, §5 reviews related work, §6 gives applications, and §7 discusses limitations and extensions before §8 concludes.

2 A Three-Layer Decomposition

We model any AI4MATH system as the composition of three layers, depicted in Figure 1. Each layer has a different artifact, a different notion of correctness, and a different failure structure.

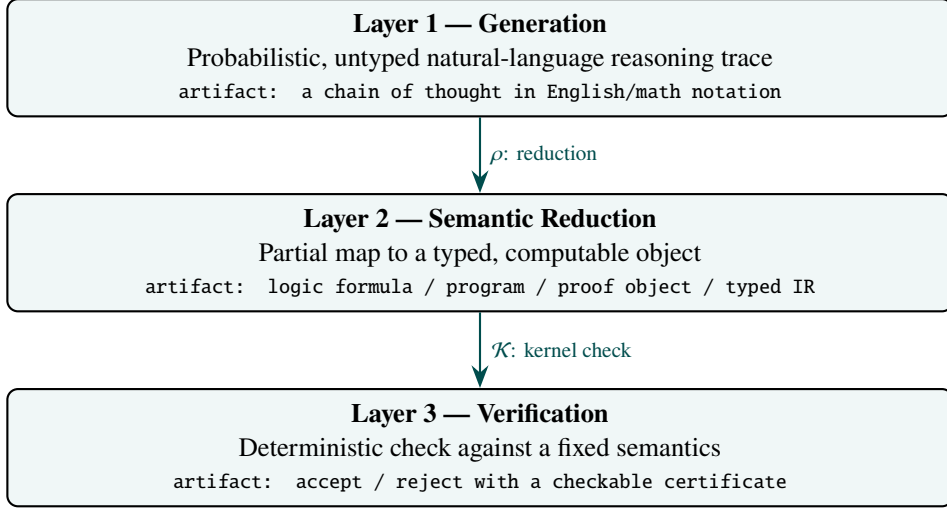


Figure 1: The three-layer decomposition. Generation produces informal text; semantic reduction converts it to a typed object; verification checks that object. The two arrows are qualitatively different maps: the first is partial and meaning-sensitive, the second deterministic and mechanizable.

2.1 Layer 1: Generation

The generation layer maps a problem q to a natural-language reasoning trace t :

$$g_{\theta} : q \mapsto t = (s_1, s_2, \dots, s_k), \quad (1)$$

where each s_i is a sentence mixing prose and notation. This is what chain-of-thought prompting elicits (Wei et al., 2022), and what Minerva-style models produce at scale (Lewkowycz et al., 2022). Three properties matter here. The map is *probabilistic*: re-prompting yields different traces. It is *untyped*: sentences carry no machine checkable declaration of what kind of object each symbol denotes. And it is *semantically open*: phrases such as “it follows that” or “obviously” do not specify which rule was applied or which prior statements license the step. These properties make generation flexible but leave validity as a property a human reader must judge.

2.2 Layer 2: Semantic reduction

The reduction layer attempts to attach a precise meaning to the trace by mapping it into a formal, computable representation:

$$\rho : t \multimap i, \quad (2)$$

where i is a typed object and the harpoon denotes that the map is *partial*: it is undefined for text it cannot interpret. Concretely, reduction targets one of four forms:

- a **logical formula** in first- or higher-order logic;
- an **executable program** in a programming language or DSL;
- a **proof object** (a term of a proof assistant’s type theory, or a sequence of tactics);
- a **typed intermediate representation** such as an annotated AST or constraint graph.

The theoretical foundation for proof objects is the *Curry–Howard* correspondence, under which propositions are types and proofs are inhabiting terms (Sørensen and Urzyczyn, 2006). Program-of-thoughts prompting reduces to executable programs (Chen et al., 2023); autoformalization reduces natural-language statements to Lean or Isabelle terms (Polu et al., 2022; Xin et al., 2024); the DSL used by AlphaGeometry is a domain-specific typed representation (Trinh et al., 2024).

Table 1: Taxonomy of reduction targets. Each target licenses a different verification guarantee and is exemplified by a cited system.

Reduction target	Formal basis	Guarantee established	Example systems
Logical formula (FOL/HOL)	proof theory	logical consequence	SMT solvers, ATP
Executable program	operational/denot. semantics	computed value or property	PoT (Chen et al., 2023), code tools
Proof object / tactics	Curry–Howard	theorem derivability	GPT-f (Polu and Sutskever, 2020), B DeepSeek-Prover (Xin et al., 2024), AlphaGeometry (Trinh et al., 2024)
Typed AST / DSL	type systems	well-typedness + constraints	symbolic solvers
Constraint graph	CSP semantics	feasibility	

2.3 Layer 3: Verification

The verification layer checks the reduced object against a fixed formal system, rather than against prose. Writing Γ for the context of axioms, definitions, and established lemmas and φ for the target proposition, the kernel decides a judgment

$$\Gamma \vdash i : \varphi, \quad (3)$$

returning accept or reject together with a certificate that can be independently replayed. Depending on the object, the check establishes *derivability* (a theorem follows from the context), *consistency* (no contradictory constraints), *factual grounding* (referenced premises exist and are applied correctly), or *constraint satisfaction* (the computed value meets the target). Tools include proof assistants with small trusted kernels—Lean (de Moura and Ullrich, 2021), Coq, Agda, Isabelle—and SMT or symbolic solvers. The defining property is that a sound kernel never accepts an invalid object *of the kind it claims to check*.

2.4 The two arrows are not symmetric

The decomposition’s content lies in the difference between its two arrows (Table 1). The reduction arrow ρ is partial, meaning-sensitive, and as hard as translation: ambiguity in the source can make multiple objects plausible. The verification arrow $\mathcal{K}$, by contrast, is a total mechanized decision over well-formed inputs—it is exactly the part that has been engineered to be trustworthy. A system that speaks only of “end-to-end verification” conflates the two and thereby hides where reliability is actually won or lost.

3 The Bottleneck Is Semantic Reduction

3.1 What the verifier actually consumes

A verifier cannot reach back into natural language. It consumes a *computable object*: a term to type-check, a program to run, a constraint set to solve. Hence verification can establish validity only conditional on reduction having built the right object:

Proposition 1 (Conditional soundness). *Let $\mathcal{K}$ be a sound kernel and let $i = \rho(t)$. Then*

$$\mathcal{K} \text{ accepts } i \wedge i \text{ faithfully encodes } t \implies t \text{ is a valid argument.} \quad (4)$$

If reduction either fails or produces an object that does not encode the intended argument, acceptance of i implies nothing about t .

The second conjunct is usually left implicit—and is where the danger is. A formal proof of the *wrong statement* is still a valid proof object; the kernel has done its job while the overall conclusion is false.

3.2 Failure modes at the reduction layer

We distinguish three recurrent reduction failures.

Ambiguity (semantic instability). The text admits more than one formal reading. “The product of consecutive numbers exceeds their sum by one” leaves the number of terms and the comparison implicit; different reductions yield different formulas, and the model may silently pick one.

Missing typing. Symbols are introduced without a sort—a quantity versus an index, a scalar versus a function, an integer versus a set. A proof step that treats a length as an angle then has no well-typed target.

Non-invertibility and free variables. The text refers to quantities never given (“some apples”, “an unspecified price”). The resulting term contains free variables and is not a closed object; no ground truth can be computed, so the map is undefined.

These are precisely the failures our reference implementation makes observable (§4.5).

3.3 An event model of (un)reliability

To make the bottleneck measurable, define four events for a produced answer:

$$R = \{\text{reduction succeeds: a well-typed object exists}\}, \quad (5)$$

$$F = \{\text{the reduced object is faithful to the intended argument}\}, \quad (6)$$

$$A = \{\text{the kernel accepts the object}\}, \quad (7)$$

$$T = \{\text{the delivered conclusion is true}\}. \quad (8)$$

A sound kernel satisfies $A \wedge F \Rightarrow T$ (Proposition 1). Two consequences follow. *False acceptance*—a delivered conclusion the system certified—can occur only when $A \wedge \neg F$: the kernel verified an unfaithful object. *Undetected error* occurs when $\neg R$: no object exists, so no certificate of any kind is possible and a hallucination is outside the verifier’s reach. Conditioning on acceptance,

$$\Pr[T \mid A] = \Pr[F \mid A], \quad (9)$$

because under a sound kernel acceptance guarantees truth exactly when the object was faithful. Reliability engineering beyond the kernel is therefore engineering of the event F —the quality of reduction—and of the coverage of R . This motivates two metrics:

$$\text{Verifiability rate } V = \Pr[R], \quad (10)$$

$$\text{False-acceptance rate } FA = \Pr[\neg T \mid A]. \quad (11)$$

Reporting accuracy without V is incomplete: an accurate answer that no one can verify contributes nothing to reliability, and a high V with low faithfulness is actively dangerous.

3.4 Why statistical verifiers do not close the gap

Learned verifiers—outcome models that judge the final answer and process models that score each step (Lightman et al., 2024; Uesato et al., 2022)—improve accuracy and are cheap at inference. They do not, however, implement Layer 3 as defined here. They return a calibrated probability over correctness rather than a judgment against a fixed semantics, they are themselves probabilistic and trained on the same distribution that produces the errors, and they can share the generator’s blind spots (for instance, pattern-matching an off-by-one answer that “looks right”). Self-consistency (Wang et al., 2023) has the same status: agreement across samples is evidence, not a certificate. These methods are best understood as *guides for search and reduction*—they decide where a formal object is worth constructing—not as substitutes for the kernel.

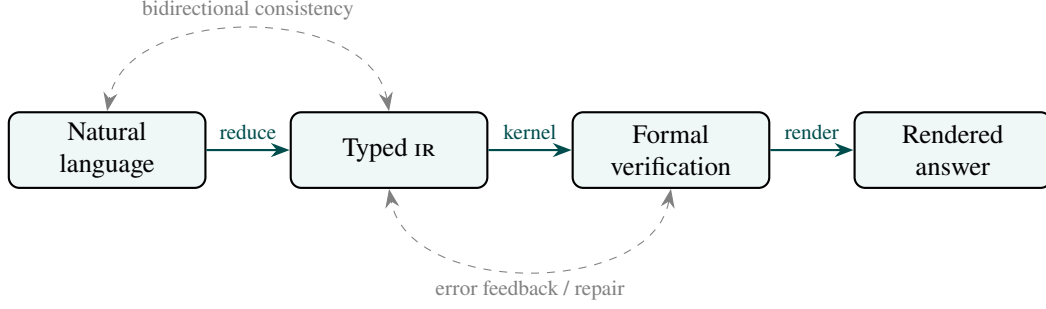


Figure 2: The proposed pipeline. Natural language is the interface; the typed IR is the execution core. Rejection produces machine-readable error feedback that drives repair, and every repaired object is re-verified from scratch rather than patched around.

4 The Framework

4.1 A typed intermediate representation

We propose making the reduction artifact explicit as a typed intermediate representation with four components:

1. a **type system** declaring the sort of every sub-object (Int , Rat , $\text{Set}[\alpha]$, $\text{Func}[\alpha, \beta]$, $\text{Proof}[\varphi]$, $\dots$);
2. an **operational semantics** fixing how objects evaluate or reduce, so two implementations cannot disagree;
3. a **constraint system** carrying side conditions (domains, ranges, well-definedness such as nonzero divisors);
4. a **compositional structure** (AST or graph) in which sub-proofs and sub-computations are independently addressable and reusable.

The type system is the key discipline: an ill-typed reduction is rejected *before* verification, converting silent semantic errors into visible type errors. This is the same mechanism that makes proof assistants trustworthy, applied at the reduction boundary.

4.2 The generate-IR-verify-render pipeline

The standard workflow is:

$$\text{Natural language} \xrightarrow{\rho} \text{Typed IR} \xrightarrow{\mathcal{K}} \text{Verification} \xrightarrow{r} \text{Rendered answer.} \quad (12)$$

The key idea is a strict division of labor: prose is generated and rendered for human communication, but all claims *live* as IR objects, and only kernel-accepted objects are rendered as conclusions. Crucially, rejection is not a dead end. The kernel emits a precise error (a type mismatch, an unresolved premise, a failed constraint) that is fed back to the generator; this is the mechanism shown effective by proof repair in Baldur (First et al., 2023) and by tactic feedback in provers (Xin et al., 2024). Every repaired object is re-verified *from scratch*, so a repair cannot mask an earlier error.

4.3 Bidirectional consistency

Reduction has a companion rendering map $r : i \mapsto t'$ that turns a verified object back into prose. We require a round-trip condition:

$$r(\rho(t)) \equiv t, \quad (13)$$

where $\equiv$ denotes semantic equivalence of the text, not literal equality. This serves two purposes. *Interpretability*: a reader can inspect, for every formal step, the prose it corresponds to. *Traceability*: if

Table 2: A ten-item reliability checklist, organized by layer. Items are binary and auditable; “not applicable” answers require justification.

Layer	#	Check
Generation	1	Problems, prompts, exemplars, and model versions are archived and versioned.
	2	Multiple samples are drawn; disagreement or self-inconsistency is flagged, not averaged away.
Reduction	3	Every claim is mapped to a typed object with a declared signature.
	4	Partial reductions are explicitly marked “unverified”; no silent defaults or coercions.
	5	Round-trip rendering is checked against the source text (§4.3).
Verification	6	Objects are checked by a fixed, version-pinned, sound kernel.
	7	The complete object is checked, not only the final answer; all premises are resolved.
	8	Kernel errors drive repair; repaired objects are re-verified from scratch.
Evaluation	9	Verifiability rate V is reported alongside accuracy, including robustness across template instantiations.
	10	Code, seeds, IR logs, and certificates are released so results can be regenerated.

Table 3: Case study results from a real run (seed pinned; reproducible via the included script). All failures appear in the reduction columns; the kernel never erred on an object it could inspect.

Family	Inst.	Red. +	Red. −	Correct accepted	Wrong rejected	Unverifiable
F1 one-step	12	10	2	10	20	6
F2 two-step	12	10	2	10	20	6
F3 rate/ratio	12	10	2	10	20	6
Total	36	30	6	30	60	18

the rendered text does not match what was said, reduction silently changed the claim—the $\neg F$ event—and the mismatch is caught by a human at the boundary rather than discovered after a false acceptance. Bidirectional links are thus the practical instrument for guarding faithfulness, which the kernel alone cannot check.

4.4 A reliability checklist for AI4MATH systems

As a directly usable contribution, Table 2 distills the framework into ten checks. It is written so that a reviewer can audit a submission item by item; a system that cannot satisfy an item should say so explicitly rather than leave the gap implicit.

4.5 Reproducible reference implementation and case study

To test the claim that failures localize at reduction, we include a small reference implementation (`artifacts/reduction_case_study.py`, standard library only, pinned random seed). The domain is a controlled dialect of arithmetic word problems with three families: one-step additive, two-step give-away/earn, and rate/ratio with an aggregate discount. Each problem is reduced by an explicit parser to a typed AST (Int-saturated Num and BinOp nodes), symbolically evaluated, and then presented with three candidate answers—one correct and two wrong, including an off-by-one candidate that a statistical reader might accept. We additionally instantiate *malformed* variants containing unbounded quantities (“some apples”) or free variables (“an unspecified price”).

Table 3 reports the actual output. Of 36 problem instances, 30 reduced to closed typed objects and 6 did not; the *verifiability rate* was $V = 30/36 = 0.833$. On the 90 verifiable candidate answers (30 correct, 60 wrong), the kernel accepted every correct answer and rejected every wrong one—including all off-by-one candidates—a classification rate of 1.000. The 18 candidates attached to malformed problems were neither accepted nor rejected: with no object to check, they were simply *unverifiable*, the $\neg R$ regime.

Observation 1. *In the controlled dialect, every observed failure (6/6 reduction failures, 18/18 unverifiable candidates) lies at Layer 2; Layer 3 made no errors on its defined inputs and Layer 1 outputs were harmless once reduction pinned their meaning.*

Boundaries of the evidence. This is a proof-of-concept on a templated dialect, not a benchmark result: the parser is hand-written, the semantics are elementary integer arithmetic, and candidate generation is simulated rather than produced by an LLM. The experiment supports the structural claim—that given a faithful reduction, verification is deterministic, and that without it no verification is possible—but says nothing about how often contemporary LLMs achieve faithful reduction at scale, which is an empirical question for which (Mirzadeh et al., 2024) and formal proving results such as (Hubert et al., 2025) provide complementary evidence.

5 Related Work

Reasoning via prompting. Chain-of-thought prompting demonstrates that intermediate steps improve reasoning and that the ability emerges at sufficient scale (Wei et al., 2022); self-consistency replaces greedy decoding with marginalization over sampled paths (Wang et al., 2023). These methods operate entirely in Layer 1 and therefore inherit its unverifiability; in our framing they shape the distribution of traces but produce no object a kernel can check.

Tool use and program-based reasoning. Toolformer shows that models can self-supervise API calls (Schick et al., 2023); program-of-thoughts prompting moves computation into an interpreter, with the LLM responsible only for the program (Chen et al., 2023); Minerva fine-tunes a large model on technical content and formats steps in $\text{\LaTeX}$ (Lewkowycz et al., 2022). These are instances of reduction to *executable programs* (row 2 of Table 1). Our framework makes explicit what they assume implicitly: the program, not the prose, is the claim, and the interpreter plays the kernel for the computational fragment while the reasoning connecting programs still needs its own reduction.

Statistical verification. Outcome and process supervision train verifiers on final results or individual steps (Uesato et al., 2022); the process-supervised results of Lightman et al. (2024) show strong gains on *MATH*. As argued in §3.4, these are probabilistic oracles that guide search rather than sound checkers; they fit most naturally *inside* the pipeline as proposals for what to reduce and verify formally.

Formal theorem proving with learned components. GPT-f trains a transformer on Metamath and contributed the first deep-learning proofs accepted into a formal library (Polu and Sutskever, 2020); expert iteration with statement curriculum learning extends this to olympiad problems (Polu et al., 2022), evaluated on the cross-system miniF2F benchmark (Zheng et al., 2022). Baldur generates whole proofs and repairs them using the proof assistant’s error messages (First et al., 2023); LEGO-Prover grows a library of verified, retrievable skills and solves roughly half of miniF2F test (Wang et al., 2024); LeanDojo provides open tooling, fine-grained premise annotations, and a retrieval-augmented prover (Yang et al., 2023); DeepSeek-Prover shows that large-scale synthetic Lean data yields strong results from a small open model (Xin et al., 2024); AlphaProof combines a proof-search policy with test-time reinforcement learning and reached the silver-medal level at IMO 2024, with every proof Lean-verified (Hubert et al., 2025). These systems demonstrate that the verify-once-the-object-exists half of the problem is solved in

practice. Our contribution is to locate the remaining difficulty in the informal-to-formal map and to give a uniform architecture and audit instrument for it.

Neuro-symbolic geometry and symbolic mathematics. AlphaGeometry is a neuro-symbolic system in which a neural model guides a symbolic deduction engine over a custom DSL, solving 25 of 30 recent olympiad geometry problems (Trinh et al., 2024); AlphaGeometry2 extends the language to movements, linear angle/ratio/distance relations, and non-constructive problems, surpassing the average gold medalist (Chervonyi et al., 2025). Lample and Charton (2020) show that sequence models can outperform commercial algebra systems on symbolic integration and differential equations. Both lines reduce a fragment of mathematics to a typed, executable form—exactly the move we generalize— while their domain-specific reduction rules also illustrate the cost and the coverage limits discussed next.

6 Applications

Automated theorem proving. The natural instantiation is proof search with an LLM: the model proposes a proof sketch, each step is reduced to a tactic term or proof object, and the Lean/Coq kernel checks it; rejected steps return type errors that focus the next round of search (First et al., 2023; Hubert et al., 2025). The framework predicts where effort pays off: improving premise selection and autoformalization (Layer 2) raises V and faithfulness, whereas sampling more prose once reduction is saturated does not.

Mathematical problem solving. For word problems, reduction targets a symbolic program: text becomes a typed expression, a solver or interpreter evaluates it, and only the verified value is returned (Chen et al., 2023; Lample and Charton, 2020). Template robustness testing (Mirzadeh et al., 2024) becomes a Layer 2 diagnostic: a problem family is reliable only if reduction succeeds and yields equivalent objects across numerical instantiations, and any instantiation that fails reduction is reported rather than answered.

Teaching and assessment. In an educational setting, a student’s solution can be reduced to a typed object and graded against the kernel’s judgment rather than compared by surface similarity to a reference answer. Partial reductions give precise, localized feedback (“this step has no well-typed justification here”), and bidirectional rendering lets students read their own argument back in canonical form. The same checklist that audits research systems doubles as a rubric for what students should learn to demand of an AI.

7 Discussion

7.1 Limitations

We state five limitations that bound the claims. First, *not all mathematics is formalizable at acceptable cost*: designing the IR and writing the reduction rules is itself expert labor, and frontier research often lacks a settled language to reduce into—the “cold start” problem visible even in large systems (Hubert et al., 2025). Second, *reduction can be wrong while looking valid*: a well-typed object may encode a subtly different statement (the $\neg F$ event), and only bidirectional consistency and human review catch this. Third, the case study is deliberately tiny and its numbers describe the implementation, not the field. Fourth, *kernel trust has its own roots*: the soundness guarantee is relative to the kernel’s implementation and specification, which must themselves be audited. Fifth, statistical and formal methods are complementary rather than ordered: abandoning learned verifiers would forfeit cheap guidance about where formal checking is worthwhile.

7.2 Extensions

The architecture extends naturally beyond single problems. Multi-step agent systems can be required to emit a typed object at every action, so that an entire workflow—not just its final answer—is verifiable and replayable, with the constraint graph recording dependencies between steps. The same discipline supports *library learning*: verified objects accumulate as reusable, typed lemmas in the spirit of (Wang et al., 2024), and process models can be trained *from kernel decisions* rather than human labels, giving statistical guides a sound signal source. We are deliberately silent here about particular agent frameworks; the extension is a design principle, not a claim about a named system.

8 Conclusion

We argued that the reliability of AI in mathematics is decided neither at generation nor at the final checker but between them: at the *semantic reduction* that converts informal language into a typed, computable object. A three-layer decomposition makes this visible; a typed-IR pipeline with bidirectional consistency makes it engineerable; a ten-item checklist makes it auditable; and a reproducible case study shows the predicted pattern—verification is perfect on what it can see, and every failure is a failure to give it something to see. The practical maxim is compact: *reliable AI reasoning is generation plus semantic reduction plus verification, and the reduction is where the work is*. Treating that layer as a first-class component, rather than an implicit prelude to checking, is the small change in perspective we offer toward systems whose mathematical conclusions can actually be trusted.

References

- Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research (TMLR)*, 2023.
- Yuri Chervonyi, Trieu H. Trinh, Miroslav Olšák, et al. Gold-medalist performance in solving olympiad geometry with AlphaGeometry2, 2025.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021.
- Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28, Lecture Notes in Computer Science*, volume 12699, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_36.
- Emily First, Markus N. Rabe, Talia Ringer, and Yuriy Brun. Baldur: Whole-proof generation and repair with large language models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2023)*, pages 1229–1241, 2023. doi: 10.1145/3611643.3616243.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1 (NeurIPS 2021)*, 2021.
- Thomas Hubert, Rishi S. Mehta, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 2025. doi: 10.1038/s41586-025-09833-y.
- Guillaume Lample and François Charton. Deep learning for symbolic mathematics. *The Eighth International Conference on Learning Representations (ICLR 2020)*, 2020.

- Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, Vinay Misra, et al. Solving quantitative reasoning problems with language models. *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, pages 12378–12391, 2022.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024.
- Seyed-Iman Mirzadeh, Keivan Alizadeh, Hooman Shahrokhi, Oncel Tuzel, Samy Bengio, and Mehrdad Farajtabar. GSM-Symbolic: Understanding the limitations of mathematical reasoning in large language models, 2024.
- Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving, 2020.
- Stanislas Polu, Jesse Michael Han, Kunhao Zheng, Mantas Baksys, Igor Babuschkin, and Ilya Sutskever. Formal mathematics statement curriculum learning. In *The Tenth International Conference on Learning Representations (ICLR 2022)*, 2022.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.
- Morten Heine Sørensen and Paweł Urzyczyn. *Lectures on the Curry-Howard Isomorphism*, volume 149 of *Studies in Logic and the Foundations of Mathematics*. Elsevier, 2006. doi: 10.1016/S0049-237X(06)80499-4.
- Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024. doi: 10.1038/s41586-023-06747-5.
- Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback, 2022.
- Haiming Wang, Huajian Xin, Chuanyang Zheng, Lin Li, Zhengying Liu, Qingxing Cao, Yinya Huang, Jing Xiong, Han Shi, Enze Xie, Jian Yin, Zhenguo Li, Heng Liao, and Xiaodan Liang. Lego-prover: Neural theorem proving with growing libraries. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations (ICLR 2023)*, 2023.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, pages 24824–24837, 2022.
- Huajian Xin, Daya Guo, Zhizheng Yao, et al. Deepseek-prover: Advancing theorem proving in LLMs through large-scale synthetic data, 2024.
- Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Xingyu Zhu, Marko Vasiljevic, Jiahao Chen, Shubhang Khandelwal, Soroush Goudarzi, Sarfraz Khimani, Wenda Chu, Armando Solar-Lezama, Koushik Sen, Ion Stoica, and Dawn Song. Leandor: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, *Datasets and Benchmarks Track*, pages 21573–21612, 2023.

Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. minif2f: A cross-system benchmark for formal olympiad-level mathematics. In *The Tenth International Conference on Learning Representations (ICLR 2022)*, 2022.